

Model Development - IBM Data Analytics Professional Certificate

1. Simple Linear Regression (SLR)

Used to predict one dependent variable (Y) from one independent variable (X).

Example: Predict salary based on years of experience.

Equation: $Y = aX + b$

Use when the relationship is linear and you have one input.

Syntax:

```
from sklearn.linear_model import LinearRegression
```

```
model = LinearRegression().fit(X, y)
```

2. Multiple Linear Regression (MLR)

Used when you have more than one independent variable.

Example: Predict house price using size, location, and number of bedrooms.

Equation: $Y = a_1X_1 + a_2X_2 + \dots + a_nX_n + b$

Syntax:

```
model = LinearRegression().fit(X_multiple, y)
```

3. regplot() - Regression Plot

Visualizes the relationship between independent (X) and dependent (Y) variables.

Shows a regression line to help identify linear trends.

Syntax:

```
sns.regplot(x='feature', y='target', data=df)
```

4. residplot() - Residual Plot

Displays the residuals (errors between actual and predicted values).

Good residual plot: residuals randomly scattered around 0.

Bad plot: curved patterns or increasing/decreasing spread.

Syntax:

```
sns.residplot(x='feature', y='target', data=df)
```

5. Residual Plot Interpretation

If residuals show a pattern or change in spread (heteroscedasticity), the linear model may not be a good fit.

Example of bad fit: residuals fall sharply as X increases.

6. Distribution Plot

Used to compare actual vs predicted values visually.

Helpful in MLR to check if predicted values match actual values across the range.

Syntax:

```
sns.distplot(y_actual, hist=False, label='Actual')
```

```
sns.distplot(y_pred, hist=False, label='Predicted')
```

7. Polynomial Regression

Used for modeling non-linear relationships.

Adds polynomial terms like X^2 , X^3 , etc., to the model.

Syntax:

```
from sklearn.preprocessing import PolynomialFeatures
```

```
poly = PolynomialFeatures(degree=2)
```

```
X_poly = poly.fit_transform(X)
```

8. Feature Scaling and StandardScaler

Used to standardize features by removing the mean and scaling to unit variance.

Important for models sensitive to feature scale.

Syntax:

```
from sklearn.preprocessing import StandardScaler
```

```
scaler = StandardScaler()
```

```
X_scaled = scaler.fit_transform(X)
```

9. Pipeline

Chains preprocessing and modeling steps for cleaner code and automation.

Syntax:

```
from sklearn.pipeline import Pipeline
```

```
pipe = Pipeline([  
    ('scale', StandardScaler()),
```

```
('poly', PolynomialFeatures(degree=2)),  
('model', LinearRegression())  
])
```

10. Model Evaluation - Mean Squared Error (MSE)

MSE = Average of squared differences between actual and predicted values.

Lower MSE = better model.

Syntax:

```
from sklearn.metrics import mean_squared_error  
mse = mean_squared_error(y_true, y_pred)
```

11. Model Evaluation - R-squared (R^2)

Represents how much of the variance in Y is explained by X.

Closer to 1 = better. Negative R^2 indicates overfitting or poor model.

Syntax:

```
r2 = model.score(X, y)
```

12. Signs of a Good Model

- Residuals randomly scattered around zero
- Constant variance
- High R^2 value
- Low MSE
- Predicted values align with actual values
- No patterns in residual plot

13. When Model is Not a Good Fit

- Residuals show patterns (curves)
- Increasing/decreasing residual spread (heteroscedasticity)
- Low R^2 or high MSE
- Model underperforms in certain ranges